

CS 230 Distributed Computer Systems

Homework 1

1 The Approach

1.1 Simulation Unit Construction

Well, at first, to simulate the process, we need to allocate 3 arrays to store 3 interacting parts of the system, which are processing elements' requested memory module, the waiting time counter for each of the modules, and the priority of connection(determine the order of the process). In C, we can struct a simulator to encapsulate these three components to make it more organized, it is demonstrated here:

```
typedef struct sim {  
    int* processes; // the array of processes  
    int* memories; // processing elems' requested memory module  
    int* wait_times; // the waiting time counter for the modules  
    int* priorities; // the priority of connection  
} sim;
```

Then, we need another array to represent the memory module, this array is a little bit different from the former one and more complicated. It stores two sorts of information: the availability of each memory and the processor attached to the module. Pseudo code would be like this:

```
int memories[number_of_attached_processes][2]
```

It is kind of abstract, let me put it in this way, int this 2D array, each row stands for a memory module, and is of two columns. The first column in a row stands for the attached processor, the second column stands for whether the memory is connected, if the answer is yes, it will hold a value of 1, the other way round, hold a value of 0.

To make it more readable and understandable, we can encapsulate the 2 columns in a row into a list node. So in the manipulation of this array, we won't get confused. And this is how we do it:

```
typedef struct mem_queue {
    int attached_process;
    node* queue;
} mem_queue;
```

Well, at this stage, we can add this memory array into our simulator structure:

```
typedef struct sim {
    int* processes;
    int* wait_times;
    int* priorities;
    int* memories;
    mem_queue* queues;
} sim;
```

Now we get the basic skeleton of our simulator, we can move on to prioritize the processes.

1.2 Prioritizing the Waiting Elements

1.2.1 Simulator Revision

At first, to simulate this sorting process, we need an end condition, or there will be an infinite loop. As we can see from the instructions, the end condition is that the current average waiting time differs from previous waiting time by 0.02%. So, in order to get the average waiting time, we need to do 2 things, the first one is to count and record accumulated waiting time, the another one is two count the number of processors and memory modules. As we can see, the waiting time can be obtained in the simulator's waiting time array. As for the number of processors and memory modules, we can add these two attributions in the simulator like this:

```
typedef struct sim {
    int* processes;
    int* wait_times;
    int* priorities;
    int* memories;
    mem_queue* queues;
    int process_count;
    int module_count;
} sim;
```

1.2.2 Termination Condition Implementation

To get the difference between previous and current average waiting time, we need 3 variables, which are:

```
double prev_avg = 0; // previous average waiting time
double cur_avg = 0; // current average waiting time
double diff = 1.0; // diff between prev and cur waiting time
```

We also need a counter to count requests, in the process of counting average waiting time, each loop is a request. So we can initialize it as this:

```
int request_num = 0;
```

Now the basic request loop can be implemented. Here, we need a pseudo function called `get_avg_wait_time()` to get the average waiting time of the processor in every loop, which will be implemented later. The pseudo code will be like this:

```
while(request_num++) { // increment request number every loop
    // assign cur_avg of last loop to prev_avg
    prev_avg = cur_avg;

    // get average waiting time in the current loop
    cur_avg = get_avg_wait_time(sim, request_num);

    if (prev_avg != 0) // prevent the denominator being zero
        diff = fabs((cur_avg - prev_avg) / prev_avg); // calc

    if (diff < 0.0002) break; // end loop when diff < 0.02%
}
```

Well, basically the end condition is implemented here. What we should do is recording waiting time in the simulator and implementing the pseudo code `get_avg_wait_time()` as mentioned above.

1.2.3 Recording the Waiting Time

When should we increment the waiting time? Obviously, the waiting time of a process increments when it failed to access the memory module. So here is the implementation of what we would do when the processor cannot access the memory module:

```

int process_idx = 0; // start iteration from the first process
while (process_idx++) { // iterate all processes in the sim

    // get the memory queue inside the sim
    mem_queue *mem_q =
    &(sim->queues[sim->processes[process_idx]].queue);

    // check if the memory is available
    if (sim->memories[sim->processes[process_idx]] == 0 ||
    mem_q == NULL ||
    mem_q->attached_process == -1 || // initial value
    mem_q->attached_process == process_idx) {

        /* if available,
           attach the process to the memory module
           and generate a new request */

    } else {

        // increment wait time of the current process
        sim->wait_times[process_idx]++;

        /* if the process is not in the memory queue,
           add it to the memory queue */
        if(!contains(mem_q, process_idx)){
            if (mem_q->queue == NULL) {
                node* new_node = (node*) malloc(sizeof(node));
                new_node->process = process_idx;
                new_node->next = NULL;
                mem_q->queue = new_node;
            } else {
                push(&(mem_q->queue), process_idx);
            }
        }
    }
    if (process_idx == sim->process_count) break;
}

```

1.2.4 Calculation of Average Waiting Time

To calculate the average waiting time, we should add up all processors' waiting time, which is stored in the `wait_times` array inside the `sim`. My general idea is iterating the whole array and add all elements up, then divide it by the `process_count` attribution stored in the `sim`. We can implement it like this:

```
double get_avg_wait_time(sim* sim, int requests) {
    double average = 0;
    int i;

    for(i = 0; i < sim->process_count; i++){
        average += (double)((sim->wait_times[i]) / requests);
    }

    average /= sim->process_count;
    return average;
}
```

1.3 Simulation on 2 Kinds of Workload

Now we are going to simulate 2 kinds of workload, which are uniform and gaussian distribution, based on the mechanism we implemented above.

1.3.1 Uniform Distribution

To simulate the workload of uniform distribution, we need to implement a function to generate a uniform distributed random number, then populate generated numbers into the process array in the simulator. So at first, the algorithm to generate the number will be like this:

```
int gen_uniform_num(int min, int max){
    unsigned int num = (rand() % (max - min)) + min;
    return num;
}
```

Then, to populate the processor array with uniform distributed number, we should iterate the whole array like this:

```

int i = 0;
for (i = 0; i < sim->process_count; i++) {
    sim->processes[i] = gen_uniform_num(0, sim->module_count)
        % sim->module_count;
}

```

With this, a batch of uniform distributed memory request can be generated.

1.3.2 Gaussian Distribution

To generate a batch of gaussian distributed number, we also need an algorithm to accomplish it. Which is:

```

static double gen_gaussian_num(double mean, double sigma){
    double x = (double) random() / RAND_MAX;
    double y = (double) random() / RAND_MAX;
    double z = mean + (sqrt(-2 * log(x))
        * cos(2 * M_PI * y)
        * sigma);

    return z;
}

```

As to generating a batch of gaussian distributed processors, it got a little bit more complicated than uniform distribution. Because as the instruction said, later on we have to generate new request if a processor successfully connected to its requested memory module, to generate new request(add new processor) with gaussian distribution feature, we need two variables: mean and sigma. It requires an extra array to store one of the variable. Here, we choose to construct an extra array of means:

```

int* p_means = (int*) malloc(sim->process_count * sizeof(int));

```

Then, we need to assign a sigma value, it will be utilized in the gaussian formula. Let's make the number of module divided by 5.0.

```

double sigma = (double) (sim->module_count) / 5.0;

```

Now we can generate a batch of gaussian distributed processors:

```

int i = 0;
for (i = 0; i < sim->process_count; i++) {
    p_means = gen_gaussian_num(0, sim->module_count);
    sim->processes[i] = (int)
                        (gen_uniform_num(0, sim->module_count) %
                         sim->module_count);
}

```

1.4 Generating New Processor

As we can see, in the part **1.2.3**, inside the loop, if the processor can be attached to the memory, we left that part blank with just a few comments. It is because at that time we didn't implemented two kinds of workload. With the work done in **1.3**, we can begin to implement that part of code. There will be two situations that the processors can be uniform or gaussian distributed.

1.4.1 Generating Uniform Distributed Processors

At first, we need to assign memory module to the process, since it is attachable.

```

int mem;
mem = gen_uniform_num(0, sim->module_count) % sim->module_count;
sim->processes[process_idx] = mem;

```

In the mean time, we should update the memory queue with the information of newly attached process.

```

sim->queues[mem].attached_process = process_idx;

```

Finally, we sign the memory module to **1**, which means it is occupied.

```

sim->memories[mem] = 1;

```

1.4.2 Generating Gaussian Distributed Processors

As to the process with gaussian distribution, while compared to the process with uniform distribution, they are basically the same, while there is a slight difference in generating new process. We can implement it like this:

```

int mem;
mem = (int) (gen_gauss_num(processor_means[process_idx], sigma))
        % sim->module_count;
sim->processes[process_idx] = mem;
sim->queues[mem].attached_process = process_idx;
sim->memories[mem] = 1;

```

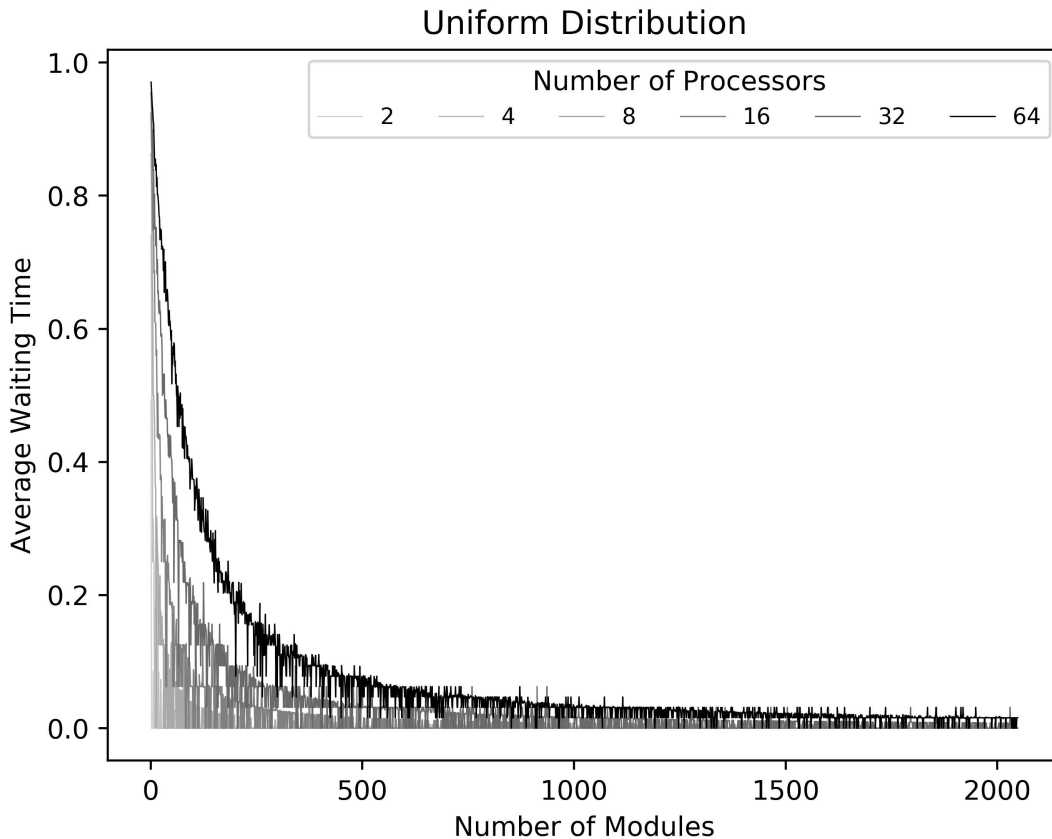
So far, we've implemented all core functions to simulate the whole procedure. Some relatively trivial parts are neglected in this report.

2 Plots and Analysis

With the simulation code, we can begin to work on the plot. At first, let's clarify the plotting conditions. We will simulate with number of processors k for $\{2, 4, 8, 16, 32, 64\}$, and number of memory modules varies from 1 to 2048.

2.1 Uniform Distribution

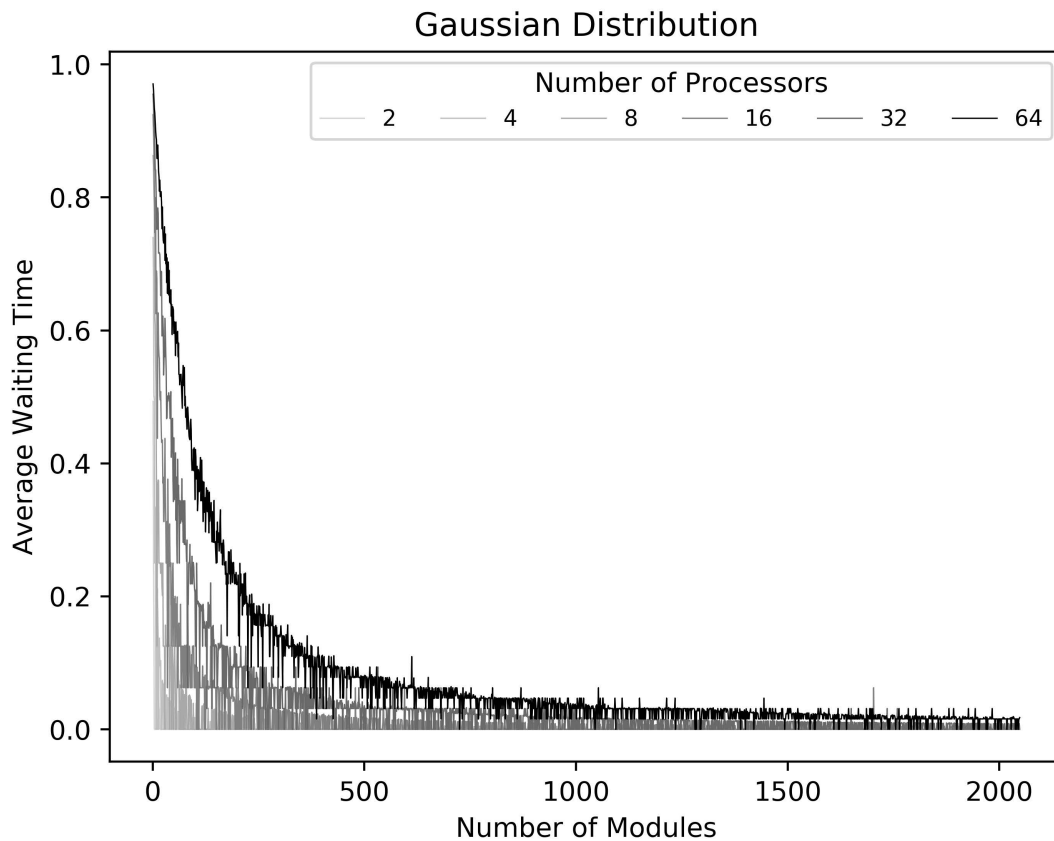
Let's see the result under the simulation of uniform distributed processors:



In this graph, I set the higher number of processors with deeper color. As we can see, when the number of processors comes lower, the distribution curve lies more closely to the two axis. As the number of memory modules increases, the average waiting time drops down dramatically, then became steadily low.

2.2 Gaussian Distribution

The result under the simulation of gaussian distributed processors is showed below:



The trend of the curve is very similar with the figure of uniform distribution processors. However, we can notice that there are two differences. One is that the curves in gaussian distribution is a little bit further away than uniform distribution, the other is that the curves in gaussian distribution have a little bit more and longer spikes, it is possibly caused by higher variation and uncertainty of processors in gaussian distribution.

3 File Listing

In *nix systems, the structure of key files of this project can be interpreted as follows:

```
includes/  
    sim.h  
    sim.c  
output/  
    gaussian.csv  
    uniform.csv  
plots/  
    gaussian.png  
    uniform.png  
main.c  
plt.py
```